

Module 8

Simple Sorting

Bubble Sort, Selection Sort, and Insertion Sort

Last time:

We looked at Queue:

- A limited data structure that has two major operations, enqueue and dequeue.
- Enqueue inserts a new element into the back of the queue and dequeue **gets and removes** the first element from the queue.
- Enqueue and dequeue time complexity is $O(1)$.
- Like Stack, it can also be used in applications as an aiding tool for a programmer.
- Java's LinkedList and ArrayDeque provide a few methods that enable programmers to use them as Queue/Deque.
- Example usage of a queue is serving people in line or queue at a coffee shop or printing jobs.

Let's shift gears for a while!

So far, we have seen some data structures: arrays, ArrayList, LinkedList, Stack, and Queue.

As we saw in Module 3, suppose that I put all your names (first name and last name) and Andrew ID or student ID in an array to have my roster.

Initially, I put them into an array without any order.

Now, it is time to grade your first homework. Oops! Well, I want to sort you by student ID or Andrew ID.

The question is “**how would you do it?**”

A computer program is NOT able to see the big picture. Thus, it needs to concentrate on the details and follow preset rules.

Three sorting algorithms in this module mainly involve two simple steps executed over and over till the items are sorted:

- Compare two items
- Swap the two items (or copy one item or items)

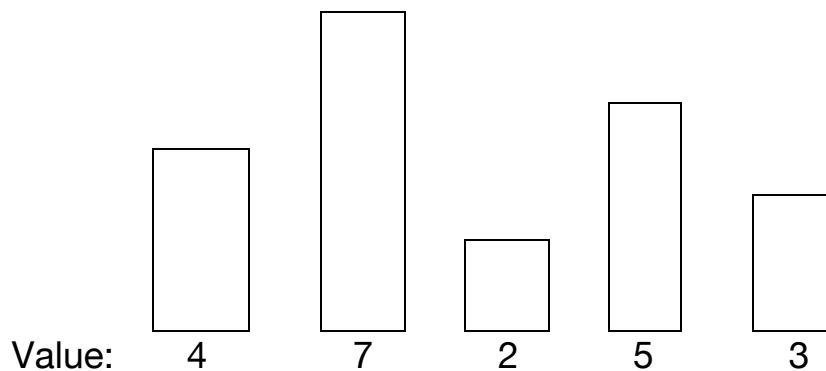
Bubble Sort: Very slow but simple

Step 1: Conceptual View

There are three basic steps in bubble sort.

- Compare two values at a time.
- If the one on the left is larger, swap them to ***BUBBLE UP the larger value to the right.***
- Move one position to the right.

Initial state



After first round

Number of comparisons? _____

Number of swaps? _____

After second round

Number of comparisons? _____

Number of swaps? _____

Now, do you see a pattern?

After third round

Number of comparisons? _____

Number of swaps? _____

After fourth round

Number of comparisons? _____

Number of swaps? _____

Step 2: Implementation View

```
int[] data = { 4, 7, 2, 5, 3 };
```

Swap method

```
// a helper method that swaps two values in an int array
private static void swap(int[] data, int one, int two) {
    int temp = data[one];
    data[one] = data[two];
    data[two] = temp;
}
```

Comparison for one round

```
// go forward from 0 to the end of the array
for (int in = 0; in < _____; in++) {
    if (data[in] > data[in+1]) { // if the left value is bigger
        swap(_____, _____, _____); // then swap
    }
}
```

Multiple rounds

```
// go backward from the last index till _____
for (int out = _____; out >= _____; out--) {
    for (int in = 0; in < _____; in++) {
        if (data[in] > data[in+1]) { // if left value is bigger
            swap(_____, _____, _____);
        }
    }
}
```

Finally, putting all the pieces together

```
public static void bubbleSort(int[] data) {
    for (int out = _____; out >= _____; out--) {
        for (int in = 0; in < _____; in++) {
            if (data[in] > data[in+1]) {
                swap(_____, _____, _____);
            }
        }
    }
}
```

Results (Let's trace!)

First round (out: _____)

4,7,2,5,3 (in: _____, in+1: _____, swap?: _____)

4,2,7,5,3 (in: _____, in+1: _____, swap?: _____)

4,2,5,7,3 (in: _____, in+1: _____, swap?: _____)

4,2,5,3,7 (in: _____, in+1: _____, swap?: _____)

Second round (out: _____)

4,2,5,3,7 (in: _____, in+1: _____, swap?: _____)

2,4,5,3,7 (in: _____, in+1: _____, swap?: _____)

2,4,3,5,7 (in: _____, in+1: _____, swap?: _____)

Third round (out: _____)

2,4,3,5,7 (in: _____, in+1: _____, swap?: _____)

2,3,4,5,7 (in: _____, in+1: _____, swap?: _____)

Fourth round (out: _____)

2,3,4,5,7 (in: _____, in+1: _____, swap?: _____)

Invariants

Conditions that remain unchanged as the algorithm proceeds.

What is the invariant in bubbleSort?

Values after "out" are sorted.

Time complexity

For 5 items, there are 4 comparisons on the first pass, 3 comparisons on the second pass, and so on, which makes 10 comparisons total:

$$4+3+2+1 = 10$$

$$(n-1) + (n-2) + (n-3) + \dots + 1 = n*(n-1) / 2$$

We can say that the bubble sort algorithm makes about $n^2 / 2$ comparisons. Also, keep in mind that there are swaps: $n^2 / 4$ on average.

When is the case that we need to swap after every comparison on every pass?

Because constants don't count in Big O notation, we can conclude that bubble sort runs in $O(n^2)$ time.

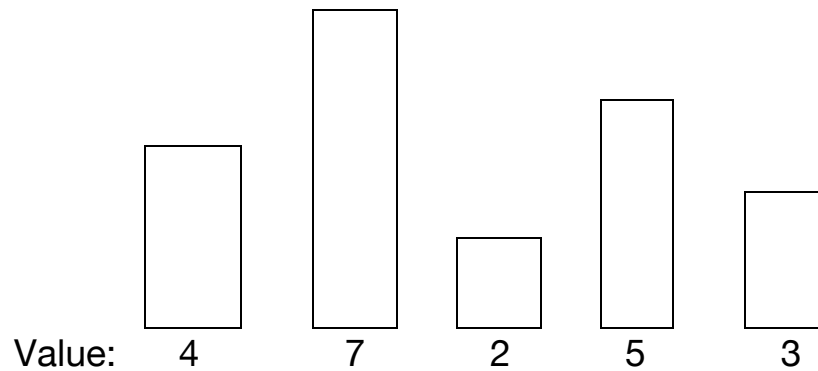
Selection Sort: Faster than Bubble Sort but still not enough

Step 1: Conceptual View

There are two steps in selection sort:

- **Pick or SELECT** the minimum value
- Swap it with the element on the left end

Initial State



After first round

Number of comparisons? _____

Number of swaps? _____

After second round

Number of comparisons? _____

Number of swaps? _____

Now, do you see a pattern?

After third round

Number of comparisons? _____

Number of swaps? _____

After fourth round

Number of comparisons? _____

Number of swaps? _____

On each pass, where are the items that are sorted?

What was it in Bubble Sort?

Step 2: Implementation View

```
int[] a = { 4, 7, 2, 5, 3 };
```

Swap method

```
// a helper method that swaps two values in an int array
private static void swap(int[] data, int one, int two) {
    int temp = data[one];
    data[one] = data[two];
    data[two] = temp;
}
```

Select the minimum value

```
int min; // min variable
for (int out = 0; out < _____; out++) {
    min = _____; // set initial min value's index

    // Select a new minimum value's index
    for (int in = _____; in < _____; in++) {
        if (data[in] < data[min]) { // if new min value found
            min = in; // reset the new min index
        }
    }

    swap(_____, _____, min); // time to put min value.
}
```

Finally, putting all the pieces together

```
public static void selectionSort(int[] data) {
    int min; // min variable
    for (int out = 0; out < _____; out++) {
        min = _____; // set initial min value's index

        // Select a new minimum value's index
        for (int in = _____; in < _____; in++) {
            if (data[in] < data[min]) { // if new min value
                min = in; // reset the new min index
            }
        }

        swap(_____, _____, min); // time to put min value.
    }
}
```


Results (Let's trace!)

First round (out: _____, min: _____)

4,7,2,5,3 (in: _____, min: _____)

4,7,2,5,3 (in: _____, min: _____)

4,7,2,5,3 (in: _____, min: _____)

4,7,2,5,3 (in: _____, min: _____)

swap(_____, _____)

2,7,4,5,3

Second round (out: _____, min: _____)

2,7,4,5,3 (in: _____, min: _____)

2,7,4,5,3 (in: _____, min: _____)

2,7,4,5,3 (in: _____, min: _____)

swap(_____, _____)

2,3,4,5,7

Third round (out: _____, min: _____)

2,3,4,5,7 (in: _____, min: _____)

2,3,4,5,7 (in: _____, min: _____)

swap(_____, _____)

2,3,4,5,7

Fourth round (out: _____, min: _____)

2,3,4,5,7 (in: _____, min: _____)

swap(_____, _____)

2,3,4,5,7

Invariants

The elements at index less than _____ variable are sorted.

Time Complexity

First of all, do you see any improvement?

Is the number of comparisons in the selection sort the same as the bubble sort?

How about number of swaps?

What is the time complexity?

Insertion Sort: Best among the three!

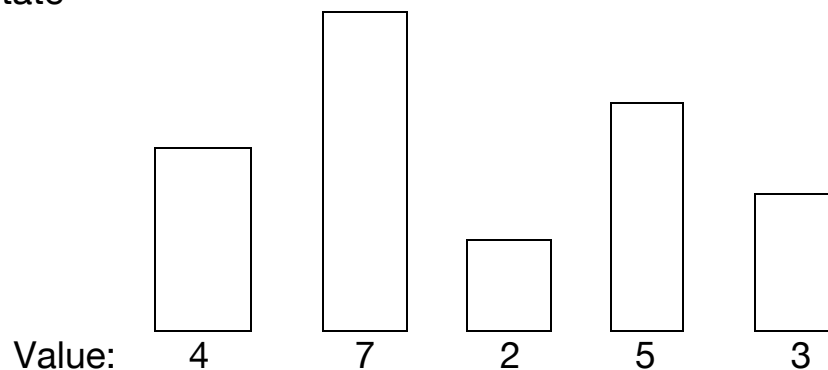
Step 1: Conceptual View

To me, this is the *most intuitive* sorting algorithm.

Most important thing in the insertion sort is that there is ***an imaginary dividing line***.

- Left-hand side of the line is sorted among themselves
- The first element of the right-hand side of the line should be inserted into the left-hand side in a proper position
 - First, we keep the value of the first element into a temp place
 - Shift the items of the left-hand side to the right so that there can be a space for the value that is stored in the temp place
 - When the position is found, ***INSERT the value into*** that position

Initial State



After first round

Where is the dividing line? _____

Which value should be kept in the temporary place? _____

After second round

Where is the dividing line? _____

Which value should be kept in the temporary place? _____

After third round

Where is the dividing line? _____

Which value should be kept in the temporary place? _____

After fourth round

Where is the dividing line? _____

Which value should be kept in the temporary place? _____

Step 2: Implementation View

```
int[] a = { 4, 7, 2, 5, 3 };
```

```
public static void insertionSort(int[] data) {
    // set and increase the dividing line
    for (int out = _____; out < _____; out++) {
        int temp = data[out]; // store the value into temp
        int in = out;

        // go backward in the left side of the imaginary line
        // to find a place to insert temp value
        while (_____ && _____) {
            data[in] = data[in-1];
            in--;
        }

        data[_____] = temp; // INSERT the temp value
    }
}
```

Results (Let's trace!)

First round (out: _____, tmp: _____)

4, ,2,5,3 (in: _____, in-1:_____, shift?:_____)

insert

4,7,2,5,3

Second round (out: _____, tmp: _____)

4,7, ,5,3 (in: _____, in-1:_____, shift?: _____)

4, ,7,5,3 (in: _____, in-1:_____, shift?: _____)

,4,7,5,3

insert

2,4,7,5,3

Third round (out: _____, tmp:_____)

2,4,7, ,3 (in: _____, in-1: _____, shift?: _____)

2,4, ,7,3 (in: _____, in-1: _____, shift?: _____)

insert

2,4,5,7,3

Fourth round (out:_____, tmp:_____)

2,4,5,7, (in: _____, in-1: _____, shift?: _____)

2,4,5, ,7 (in: _____, in-1: _____, shift?: _____)

2,4, ,5,7 (in: _____, in-1: _____, shift?: _____)

2, ,4,5,7 (in: _____, in-1: _____, shift?: _____)

insert

2,3,4,5,7

Invariants

At the end of each round, the elements less than _____ variable are sorted **AMONG THEMSELVES**.

Time Complexity

First, do you see any improvement?

Is the number of comparisons in the insertion sort the same as the bubble sort? (Hint: how far do you go back to insert each round?)

Do you notice anything different in terms of swapping?

What is the running time complexity?

There is a time that insertion sort can run even in $O(n)$ time. Can you think of it?

Now, how about the opposite case of the previous situation? Would it be faster than the bubble sort?

These three internal sorting algorithms, bubble sort, selection sort and insertion sort, all run in $O(n^2)$ time in the worst case.

However, often, insertion sort performs better than the other two because it may require a smaller number of comparisons depending on the input values and uses copying instead of swapping.